`ConcurrentLinkedDeque` is a **thread-safe, non-blocking, unbounded, double-ended queue (deque)** in `java.util.concurrent`.
It lets **multiple threads add/remove elements at both ends** *without locks*.

If `ArrayBlockingQueue` is about **backpressure + blocking**, `ConcurrentLinkedDeque` is about **high concurrency + progress without waiting**.

Mental model (what it is)

> 🛣️ A **lock-free deque**
> ↔️ Insert/remove from **head or tail**
> 🚫 No blocking, no capacity limit
> ⚡ Extremely scalable under contention

Key properties

Property	Description
Thread-safe	Safe for many threads
Non-blocking	No `lock()`, no `wait()`
Unbounded	Grows as needed
Deque	Head & tail operations
Weakly consistent iterators	No `ConcurrentModificationException`

What “non-blocking” really means

- Threads **never park or sleep**
- Operations use **CAS (Compare-And-Swap)**
- If a conflict happens → retry
- Guarantees **lock-free progress**:
  - At least one thread always makes progress

This avoids:

- Deadlocks
- Priority inversion
- Lock convoying

How it works internally

1. Doubly-linked nodes (but lock-free)

Each node roughly has:

```
prev <-> item <-> next
```

But unlike `LinkedList`, updates are done via **atomic CAS operations**, not synchronized blocks.

---

2. Head and tail pointers

- `head` → first node
- `tail` → last node
- Both are updated using **CAS loops**

If two threads race:

- One CAS wins
- The loser retries with fresh pointers

---

3. Logical vs physical removal

This is subtle and important 👇

1. **Logical removal**
    - Node's `item` is CAS'd to `null`
    - Node is now "deleted"
2. **Physical unlinking**
    - Neighbor pointers are fixed later
    - Cleanup may be done by another thread

This design:

- Keeps operations fast
- Avoids global coordination
- Trades immediacy for scalability

What happens during operations

`addFirst(e)` / `addLast(e)`

1. Create new node
2. Read current head/tail
3. CAS new node into position
4. Fix neighbor links if needed
5. Retry on failure

✔️ Never blocks
✔️ Linearizable

`pollFirst()` / `pollLast()`

1. Traverse from head/tail
2. Find node with non-null `item`
3. CAS item → `null` (logical delete)
4. Return value

✔️ Fast even under contention
❗ Might skip over logically deleted nodes

What it does *not* support

🚫 No blocking operations

- No `take()`
- No `put()`
- No capacity limits

🚫 No fairness guarantees

- Fast threads may win more often

If you need blocking → use `BlockingDeque`

Iterators: weakly consistent

- They:
  - Reflect *some* state of the deque
  - May or may not see concurrent updates
  - Never throw `ConcurrentModificationException`

This is perfect for monitoring, logging, traversal under concurrency.

When should you use ConcurrentLinkedDeque?

✅ Excellent choice when:

- Very **high contention**
- Many threads push/pop frequently
- You want **lock-free performance**
- Tasks can be added/removed from both ends
- Work-stealing algorithms

🚫 Avoid when:

- You need **backpressure**
- You must block when empty/full
- Memory growth must be bounded

Common real-world use cases

- **Work-stealing thread pools**
- **Task schedulers**
- **Event pipelines**
- **Caches with eviction at both ends**
- **High-throughput queues**

Comparison with related queues

Queue	Blocking	Bounded	Lock-free	Ends
`ArrayBlockingQueue`	✅	✅	❌	One
`LinkedBlockingDeque`	✅	Optional	❌	Two
`ConcurrentLinkedQueue`	❌	❌	✅	One
`ConcurrentLinkedDeque`	❌	❌	✅	Two

Tiny example

```
Deque<Integer> deque = new ConcurrentLinkedDeque<>();

deque.addFirst(1);
deque.addLast(2);

System.out.println(deque.pollFirst()); // 1
System.out.println(deque.pollLast());  // 2
```

One-line intuition (interview gold)

> **ConcurrentLinkedDeque is a lock-free, unbounded deque built with CAS, optimized for high concurrency, where operations never block but may retry.**

If you want next:

- Internals of **CAS & ABA problem**
- Why **LinkedBlockingDeque uses two locks**
- How **ForkJoinPool uses deques**
- Performance trade-offs vs blocking queues

Just say the word 👌